CEBU INSTITUTE OF TECHNOLOGY UNIVERSITY

COLLEGE OF COMPUTER STUDIES

Software Design Description

for

PrintFlow

[redacted]

Change History Signature

Preface

About This Document
This Software Design Description (SDD) provides a comprehensive technical blueprint for the implementation of PrintFlow. This document formally translates the functional and non-functional requirements defined in the PrintFlow Software Requirements Specification (SRS) into concrete architectural and component-level design decisions.

Intended Audience
This SDD is intended to serve as the primary reference for developers and software engineers during the implementation phase of the project. It ensures that the system's state-driven workflow, edge-computing print daemon, and multi-tenant isolation are built in strict alignment with the specified requirements.

System Overview
PrintFlow is a multi-tenant order and queue management system designed for the Cebu Institute of Technology - University (CIT-U) and its proximity print shops. Functioning as both a web and locally deployed application, the system is built to manage the end-to-end print fulfillment lifecycle for campus-adjacent print vendors and their student clients.

Document Organization
To provide a complete technical blueprint, this document covers the architectural design, detailed component design, object-oriented models, and data design for each system module. The technical specifications are divided into the following core system components:

- **Identity and Access Management (IAM):** Details user registration, authentication, and role-based access control (RBAC).
- **Client Order & Payment Portal:** Outlines the processes for order submission, asset upload, and GCash escrow verification.
- **Asset Validation Constraint Module:** Describes the automated mathematical verification of uploaded design files.
- **Edge-Execution and Fulfillment Ledger:** Details the WebSocket-triggered print daemon and analytics reporting functionalities.

Scope Limitations
Please note that this document is strictly focused on software architecture and logic. It does not cover hardware procurement, physical printer configuration, or third-party GCash integration beyond the manual receipt of verification interface.

Table of Contents

1. INTRODUCTION **6**

1.1. PURPOSE 6
1.2. SCOPE 6
1.3. DEFINITIONS AND ACRONYMS 6
1.4. REFERENCES 6

2. ARCHITECTURAL DESIGN **7**

3. DETAILED DESIGN **8**

Module 1 ..... *8*
Module 2 ..... *9*

1. Introduction

1.1. Purpose

This Software Design Description (SDD) provides a comprehensive technical blueprint for the implementation of PrintFlow, a multi-tenant order and queue management system for CIT-U and proximity print shops. This document translates the functional and non-functional requirements defined in the PrintFlow Software Requirements Specification (SRS) into concrete architectural and component-level design decisions. It is intended to serve as the primary reference for developers during the implementation phase, ensuring that the system's state-driven workflow, edge-computing print daemon, and multi-tenant isolation are built in alignment with the specified requirements.

1.2. Scope

PrintFlow is a web and locally deployed application that manages the end-to-end print fulfillment lifecycle for campus-adjacent print shops and their clients. This SDD covers the design of the following core system components:

- **Identity and Access Management (IAM)** — user registration, authentication, and role-based access control
- **Client Order & Payment Portal** — order submission, asset upload, and GCash escrow verification
- **Asset Validation Constraint Module** — automated mathematical verification of uploaded design files
- **Edge-Execution and Fulfillment Ledger** — WebSocket-triggered print daemon and analytics reporting

This document covers the architectural design, detailed component design, object-oriented models, and data design for each module. It does not cover hardware procurement, physical printer configuration, or third-party GCash integration beyond the manual receipt verification interface.

1.3. Definitions and Acronyms

API (Application Programming Interface) — The set of protocols used for communication between the frontend, backend, and edge daemon.

Edge Daemon — A lightweight background script deployed on the print shop's local machine that receives WebSocket signals from the cloud backend and pushes files to the OS print spooler.

JWT (JSON Web Token) — A signed, stateless authentication token embedded with the user's Role ID, used to authorize API requests.

Multi-tenant — A system architecture where a single backend instance serves multiple distinct print shop vendors with strict data isolation between them.

RBAC (Role-Based Access Control) — A method of restricting system access based on the authenticated user's assigned role (Student, Clerk, or Manager).

RBAC Matrix — The permissions table that maps each user role to its legally accessible API endpoints.

SRS (Software Requirements Specification) — The preceding document that defines the functional and non-

functional requirements for PrintFlow.

SUS (System Usability Scale) — The evaluation metric used to measure end-user satisfaction of the application.

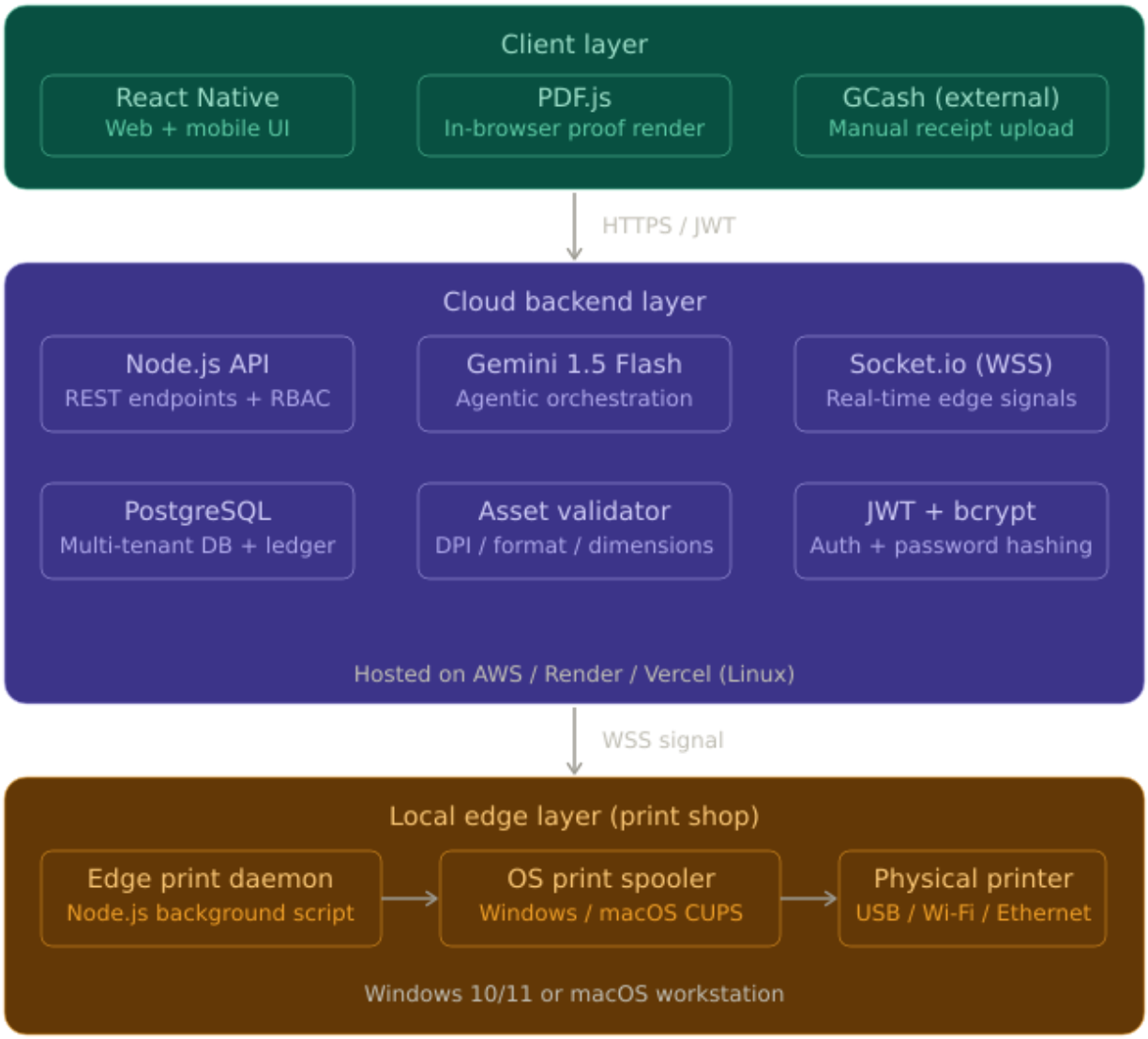
State Machine — The mathematical model governing the legal progression of a print order through defined states (e.g., OPEN_INQUIRY → PROOF_APPROVED → IN_PRODUCTION → PRINTED).

WSS (WebSocket Secure) — The encrypted, persistent communication protocol used between the cloud backend and the Local Edge Daemon.

1.4. References

2. Architectural Design

Block diagram showing all components of the system including frameworks, libraries, APIs and other technologies used in the front and back ends.

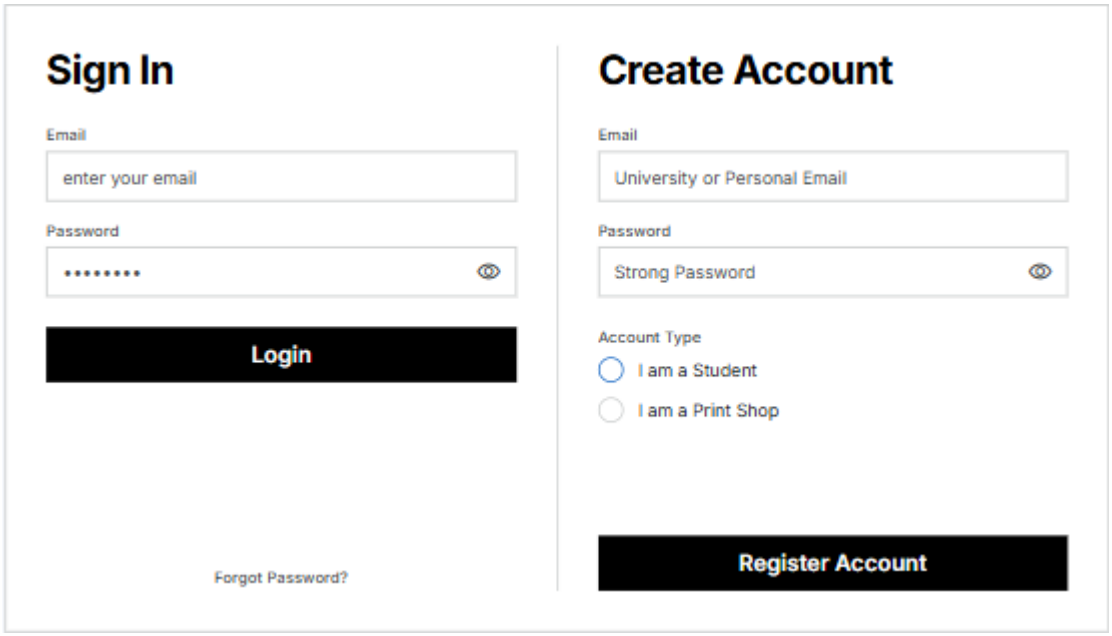


3. Detailed Design

Module 1 – Identity and Access Management (IAM)

1.1 User Registration and Role Assignment

- User Interface Design



- Front-end component(s)
    - RegistrationForm
        - Description and purpose: The root component of the Create Account panel. It manages the local form state for email, password, and account type selection. On submission, it triggers client-side validation and dispatches the registration request to the backend API.
        - Component type: React Native functional component

○ EmailInput

▪ Description and purpose: A controlled text input field that captures the user's university or personal email address. It performs real-time format validation on change and displays an inline error message if the email format is invalid.

▪ Component type: React Native TextInput

○ AccountTypeSelector

▪ Description and purpose: A radio button group that allows the Guest User to select between two account types — Student or Print Shop. The selected value is passed up to the RegistrationForm state and embedded in the API request payload.

▪ Component type: React Native TouchableOpacity group

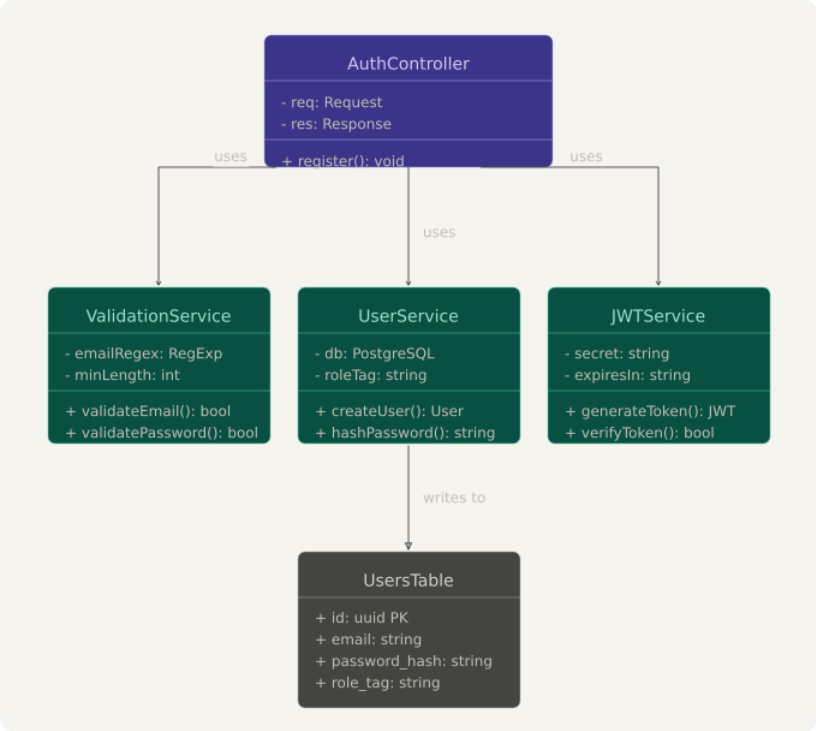
○ SubmitButton

- Description and purpose: A pressable button that triggers the RegistrationForm's submission handler. It enters a disabled/loading state while the API request is in-flight to prevent duplicate submissions.
- Component type: React Native TouchableOpacity
    - Back-end component(s)
        - AuthController
            - Description and purpose: The primary controller that handles incoming POST requests to the `/auth/register` endpoint. It receives the registration payload (email, password, account type), delegates validation and persistence to the appropriate services, and returns a signed JWT upon successful registration.
            - Component type: Node.js Express controller
        - AuthController

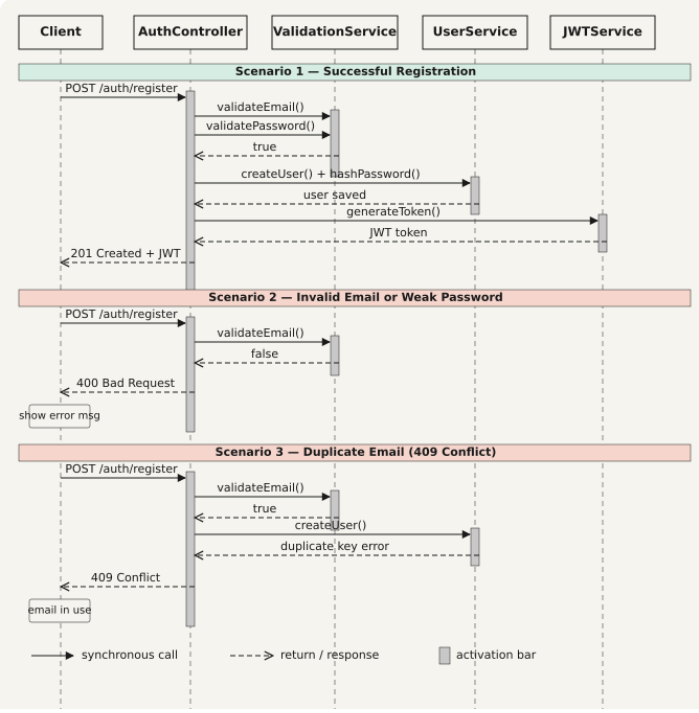
- Description and purpose: The primary controller that handles incoming POST requests to the `/auth/register` endpoint. It receives the registration payload (email, password, account type), delegates validation and persistence to the appropriate services, and returns a signed JWT upon successful registration.
- Component type: Node.js Express controller
        - ValidationService
            - Description and purpose: A utility service invoked by the AuthController to mathematically verify that the submitted email conforms to standard RFC format and that the password meets the system's minimum strength requirements (length, character complexity). Returns a structured error object if either check fails.
            - Component type: Node.js service module
        - UserService
            - Description and purpose: Responsible for creating the new user entity in the PostgreSQL database. It receives the validated credentials, invokes bcrypt to hash the password before storage, assigns the appropriate RBAC role tag (Student or Print Shop) based on the account type selection, and writes the complete user record to the Users table.
            - Component type: Node.js service module
        - JWTService

- Description and purpose: Generates a signed JSON Web Token upon successful user creation. The token payload embeds the user's unique ID and assigned Role ID, which is used by the RBAC middleware to authorize future API requests.
- Component type: Node.js service module
        - UsersTable

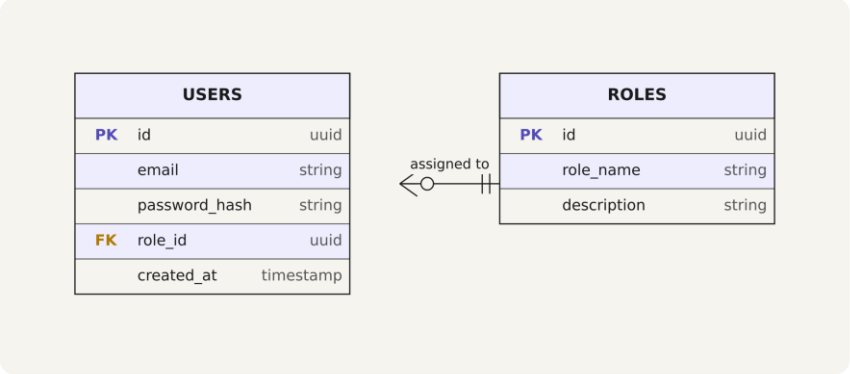
- Description and purpose: The PostgreSQL database table that persists all registered user entities. Each record stores the user ID, email, bcrypt-hashed password, role tag, and registration timestamp.
- Component type: PostgreSQL database table
    - Object-Oriented Components
        - Class Diagram



- Sequence Diagram

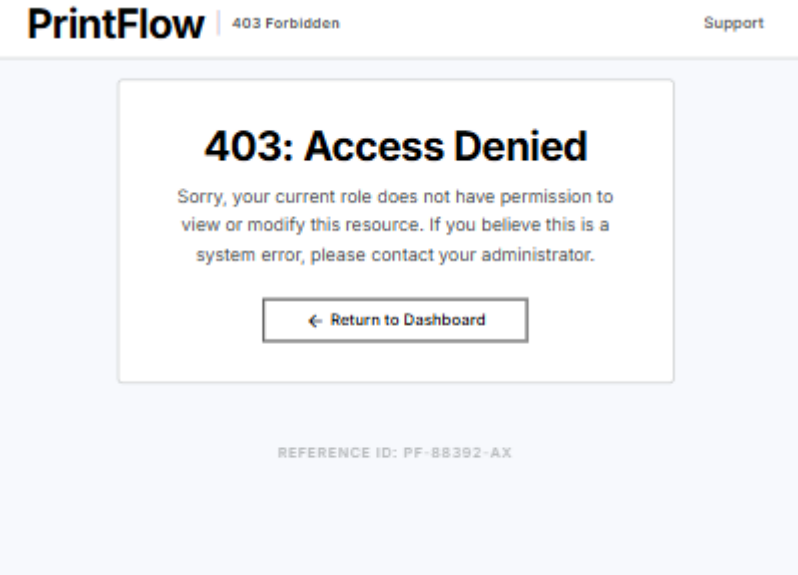


- Data Design
    - ERD or schema



1.2 Enforce Role-Based Access Control (RBAC)

- User Interface Design



- Front-end component(s)
    - ProtectedRoute
        - Description and purpose: A higher-order route wrapper that intercepts navigation attempts to any protected page. Before rendering the target component, it reads the active JWT from local storage and decodes the embedded Role ID. If the Role ID does not match the route's required permission, it redirects the user to the 403 screen.
        - Component type: React Native functional component
    - RoleGuard

▪ Description and purpose: A declarative wrapper component that accepts a `requiredRole` prop and compares it against the authenticated user's decoded role. Used inside individual screen components to conditionally render or block content at the component level, separate from route-level protection.

▪ Component type: React Native functional component

- ProtectedRoute
    - Description and purpose: A higher-order route wrapper that intercepts navigation attempts to any protected page. Before rendering the target component, it reads the active JWT from local storage and decodes the embedded Role ID. If the Role ID does not match the route's required permission, it redirects the user to the 403 screen.
    - Component type: React Native functional component

- ForbiddenScreen
    - Description and purpose: The 403 Access Denied screen rendered when a user attempts to access a route outside their RBAC scope. Displays the error code, explanation message, and a Return to Dashboard button that redirects the user to

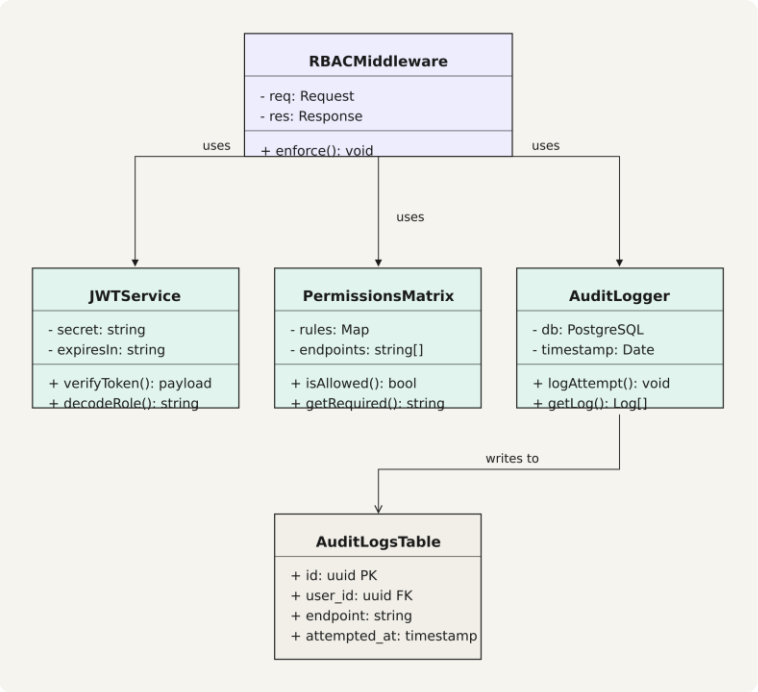
their role-appropriate home screen.

- Component type: React Native screen component

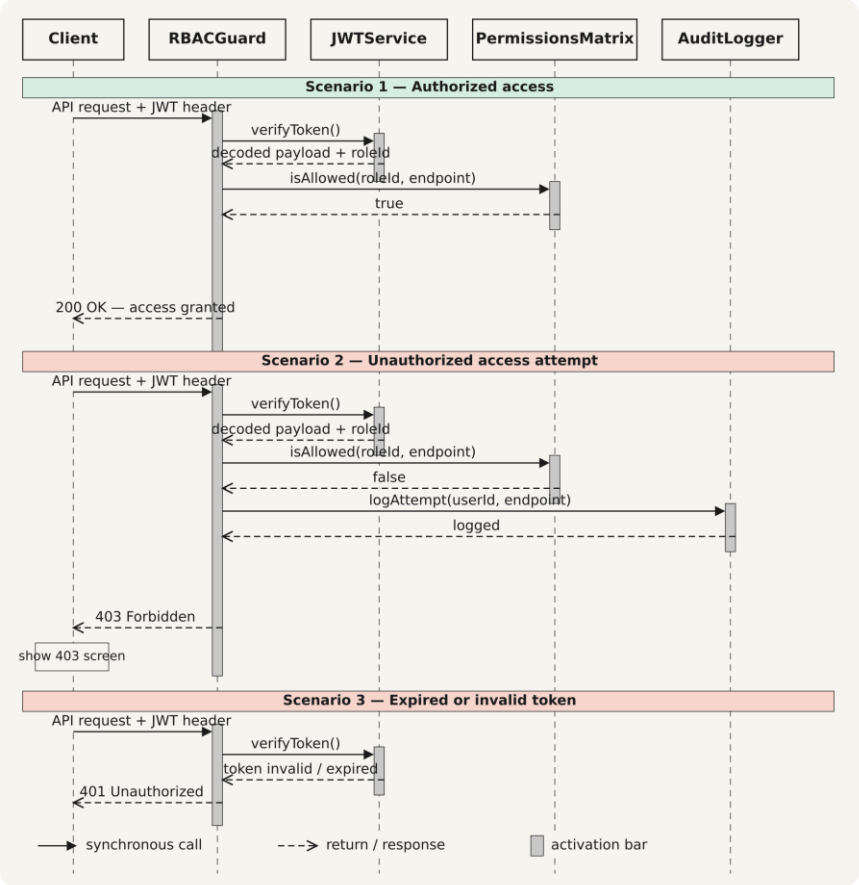
- ReturnButton
  - Description and purpose: A pressable button rendered on the ForbiddenScreen that triggers navigation back to the user's role-appropriate dashboard. The destination route is determined dynamically based on the user's decoded Role ID.
  - Component type: React Native TouchableOpacity

- Back-end component(s)
  - RBACM
    - Description and purpose: The primary middleware function attached to every protected API endpoint. It intercepts incoming requests, extracts the Authorization header, delegates token verification to JWTService, and cross-references the decoded Role ID against the PermissionsMatrix. If the role does not satisfy the endpoint's required permission, it terminates the request immediately and delegates the AuditLogger before returning a 403 response.
    - Component type: Node.js Express middleware
  - JWTService
    - Description and purpose: Reused from Transaction 1.1. In this context, it is responsible for verifying the incoming token's signature and expiry and returning the decoded payload containing the user's ID and Role ID for use by the RBACMiddleware.
    - Component type: Node.js service module
  - PermissionsMatrix
    - Description and purpose: A static configuration object that maps each protected API endpoint to its required role. The RBACMiddleware queries this object on every request to determine whether the requesting user's Role ID satisfies the endpoint's access requirement. Acts as the single source of truth for all access control rules in the system.
    - Component type: Node.js configuration module
  - AuditLogger
    - Description and purpose: A logging service invoked by the RBACMiddleware upon every unauthorized access attempt. It records the requesting User ID, the attempted endpoint, the user's Role ID, and a timestamp into the AuditLogs table in PostgreSQL for traceability and security monitoring.
    - Component type: Node.js service module
  - AuditLogsTable
    - Description and purpose: The PostgreSQL table that persists all unauthorized access attempt records. Each record stores the log ID, the User ID of the requester, the attempted endpoint, the role that was presented, and the timestamp of the attempt.
    - Component type: PostgreSQL database table

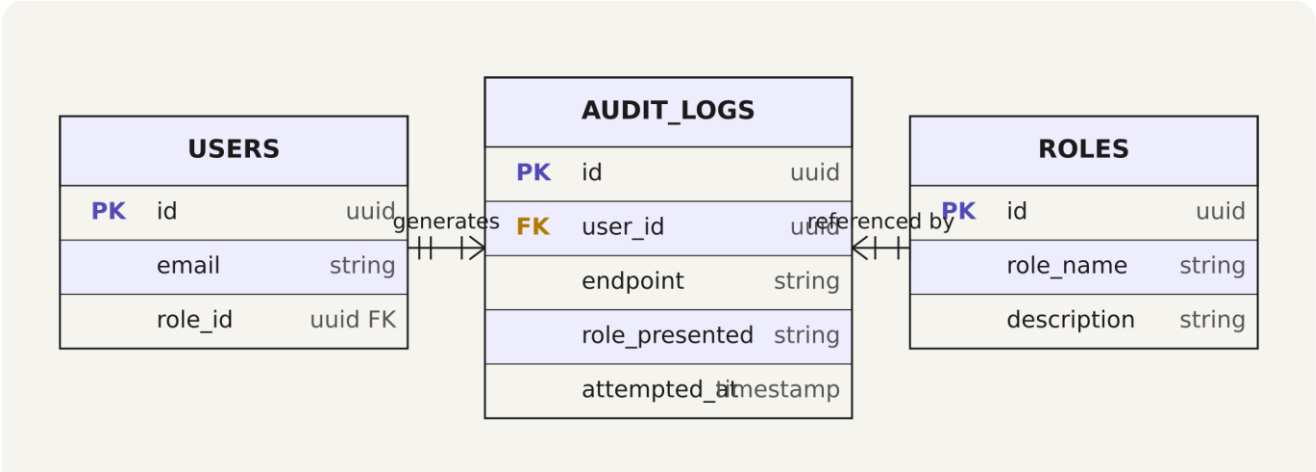
- Object-Oriented Components
    - Class Diagram



    - Sequence Diagram



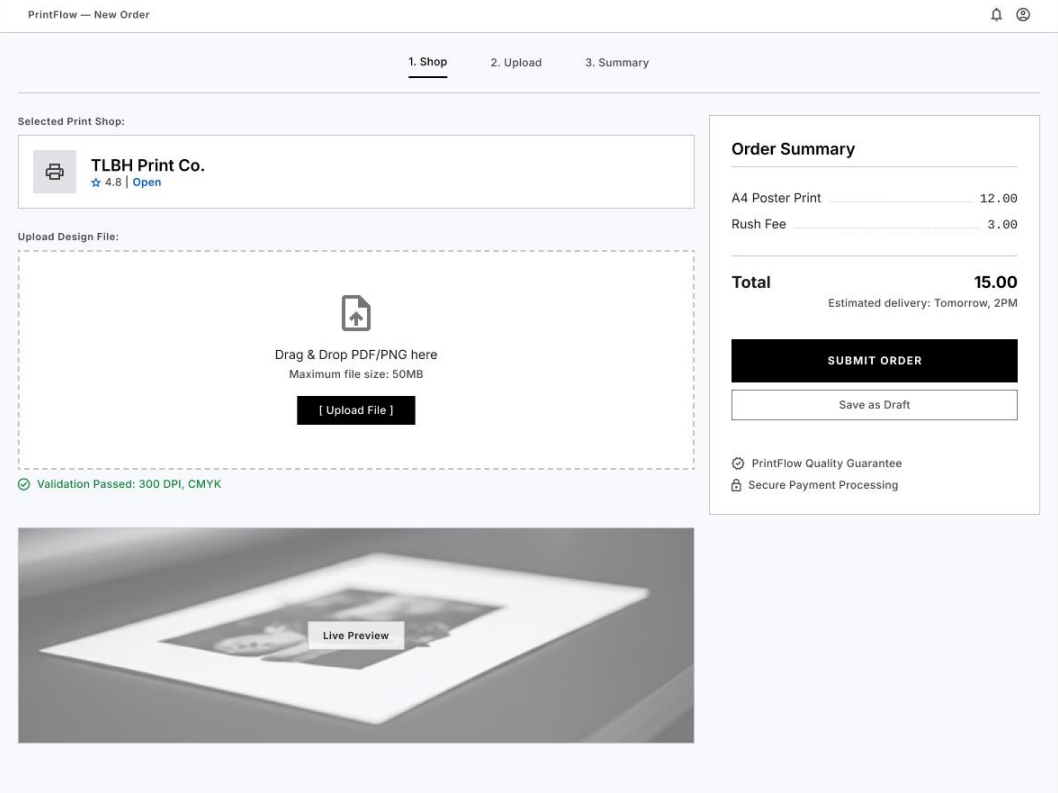
- Data Design
    - ERD or schema



Module 2 – Client Order & Payment Portal

2.1 Submit Print Order and Validate Assets

- User Interface Design



- Front-end component(s)
  - ShopSelector
    - Description and purpose: Renders the multi-tenant print shop directory as a selectable list. Each shop entry displays the shop's name, rating, and open/closed status. The selected shop's ID is stored in the form state and passed into the order payload on submission.
    - Component type: NestJS React Native functional componen
  - FileUploadZone
    - Description and purpose: A drag-and-drop upload area that accepts PDF and PNG files only. On file selection it dispatches the file to the AssetValidationService and enters a loading state while validation is in progress. Displays an inline success or error message based on the validation result.
    - Component type: React Native functional component
  - ValidationFeedback
    - Description and purpose: A conditional inline component rendered below the FileUploadZone. Displays a green success label with specific validation parameters (DPI, color profile, canvas size) if the file passes, or a red error label with the specific failure reason (e.g. "Resolution is 72 DPI, minimum 300 DPI required") if it fails.
    - Component type: React Native functional component
  - OrderSummary
    - Description and purpose: A live right-panel card that displays the itemized pricing

breakdown based on the validated file parameters and the selected shop's pricing matrix. Updates reactively as the file is validated and the shop is selected.

- Component type: React Native functional component

- SubmitButton
  - Description and purpose: A full-width pressable button that triggers the order submission handler. Remains disabled until both a shop has been selected and the uploaded file has passed validation. Enters a loading state during the API call to prevent duplicate submissions.
  - Component type: React Native TouchableOpacity

- Back-end component(s)
  - OrderController
    - Description and purpose: The primary NestJS controller handling incoming POST requests to the `/orders` endpoint. It receives the order payload (shop ID, validated asset URL, client ID) and delegates creation logic to the OrderService. Returns the created order record with its initial state of `OPEN_INQUIRY` upon success.
    - Component type: NestJS `@Controller()` with `@Post()` decorator
  - AssetValidationService

- Description and purpose: A NestJS injectable service that intercepts the uploaded file and mathematically verifies its compliance against the selected print shop's constraints. Checks file type (PDF/PNG only), DPI (minimum 300), canvas dimensions against the selected paper size, and rejects executable or macro-embedded files. Returns a structured validation result object containing either a pass confirmation with parameters or a failure with a specific error code.

- Component type: NestJS `@Injectable()` service
  - PricingService

- Description and purpose: A NestJS injectable service invoked after successful asset validation. It queries the selected shop's pricing matrix from the database and calculates the total order estimate based on the validated file parameters (paper size, DPI, color profile, quantity). Returns an itemized pricing breakdown to the frontend for display in the Order Summary panel.

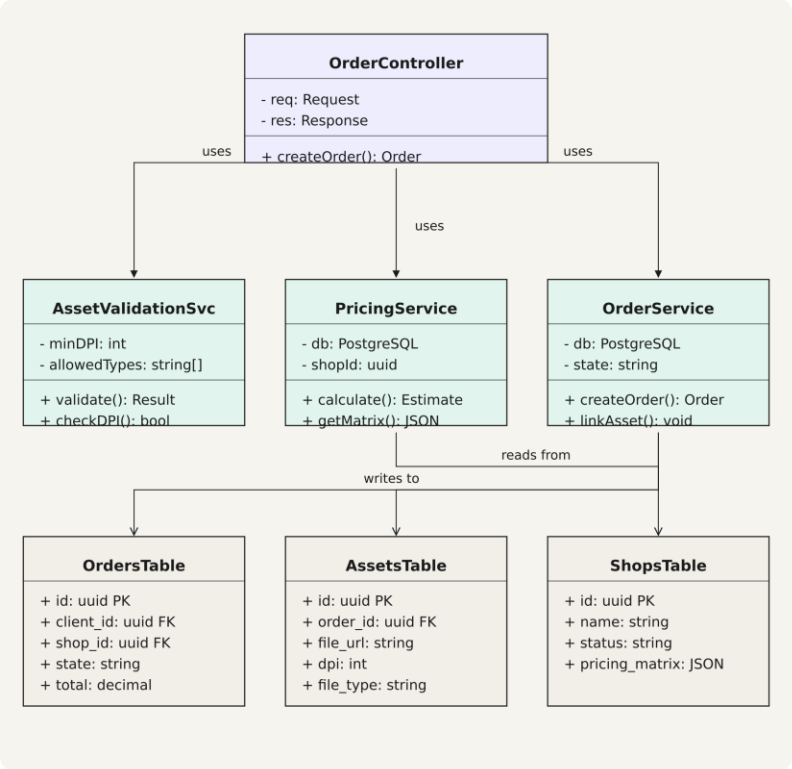
- Component type: NestJS `@Injectable()` service
  - OrderService

- Description and purpose: A NestJS injectable service responsible for persisting the new order entity to the PostgreSQL database. It receives the validated asset reference, the pricing estimate, the client ID, and the shop ID, and writes a new record to the Orders table with an initial state of `OPEN_INQUIRY`. Also links the uploaded asset to the order via the Assets table.

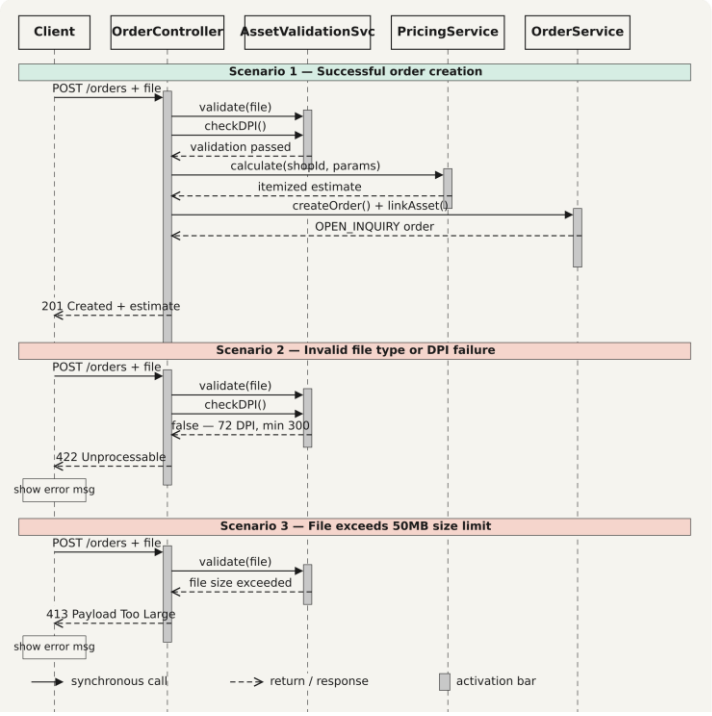
- Component type: NestJS `@Injectable()` service
  - OrdersTable
    - Description and purpose: The PostgreSQL table that persists all print order entities. Each record stores the order ID, client ID, shop ID, current state, pricing total, and timestamps for creation and last update.
    - Component type: PostgreSQL database table
  - AssetsTable

        - Description and purpose: The PostgreSQL table that stores references to all uploaded design files. Each record stores the asset ID, the linked order ID, the cloud storage URL, the validated file parameters (DPI, dimensions, file type), and the upload timestamp.
        - Component type: PostgreSQL database table
    - Component Name
        - Description and purpose: The PostgreSQL table that stores registered print shop entities and their pricing matrices. Each record stores the shop ID, shop name, availability status, and a JSON pricing matrix defining rates per paper size and service type.
        - Component type: PostgreSQL database table

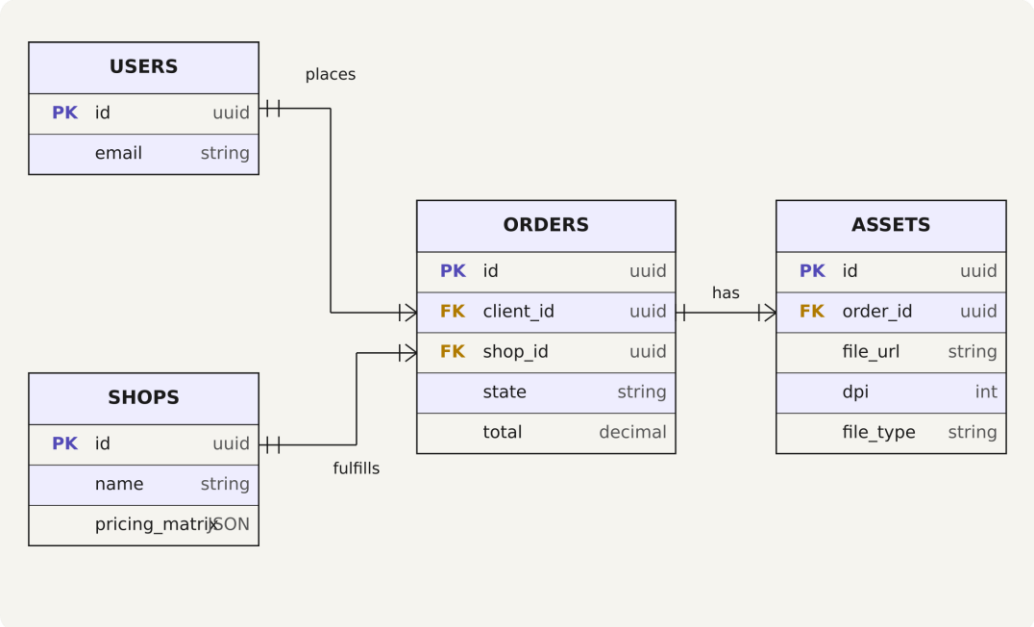
- Object-Oriented Components
    - Class Diagram



- Sequence Diagram

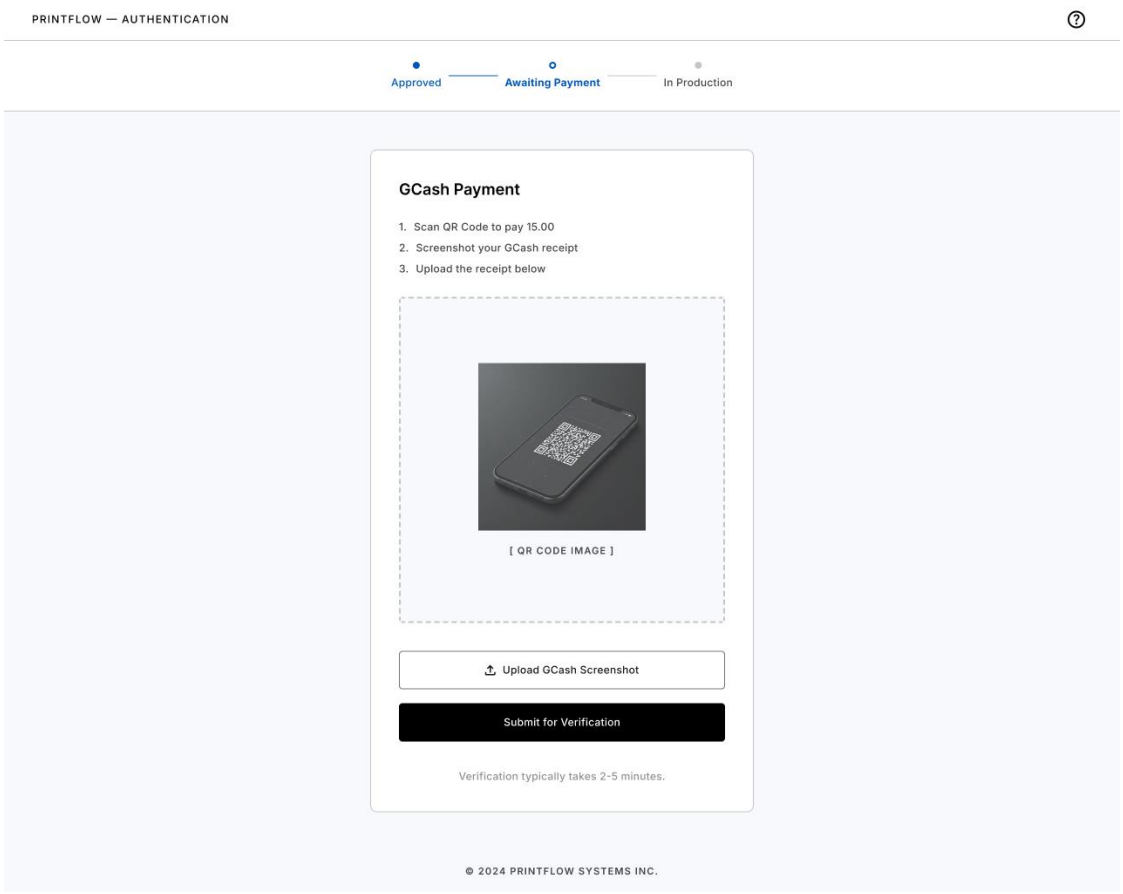


- Data Design
    - ERD or schema



2.2 Transaction name

- User Interface Design



- Front-end component(s)
    - CheckoutCard
        - Description and purpose: The root container component for the checkout screen. It manages the local state for the uploaded receipt file and the submission status. It renders the progress timeline, QR code display, upload zone, and submission button as child components.
        - Component type: React Native functional component
    - ProgressTimeline
        - Description and purpose: A horizontal stage indicator that displays the order's current position in the fulfillment lifecycle. Accepts the current order state as a prop and highlights the corresponding stage. For this transaction the active stage is `AWAITING_PAYMENT`.
        - Component type: React Native functional component
    - QRCodeDisplay
        - Description and purpose: Fetches and renders the selected print shop's GCash QR code image from the backend. Displays it inside a dashed bordered square container for the client to scan with their GCash mobile application.
        - Component type: React Native Image component

- ReceiptUploadZone
  - Description and purpose: A pressable upload area that opens the device file picker restricted to image formats only (PNG, JPG). On file selection it stores the receipt image in the CheckoutCard local state and displays a thumbnail preview to confirm the correct file was selected.
  - Component type: React Native TouchableOpacity
- SubmitVerificationButton
  - Description and purpose: A full-width pressable button that dispatches the receipt upload to the PaymentController endpoint. Remains disabled until a receipt file has been selected. Enters a loading state during the API call to prevent duplicate submissions.
  - Component type: React Native TouchableOpacity

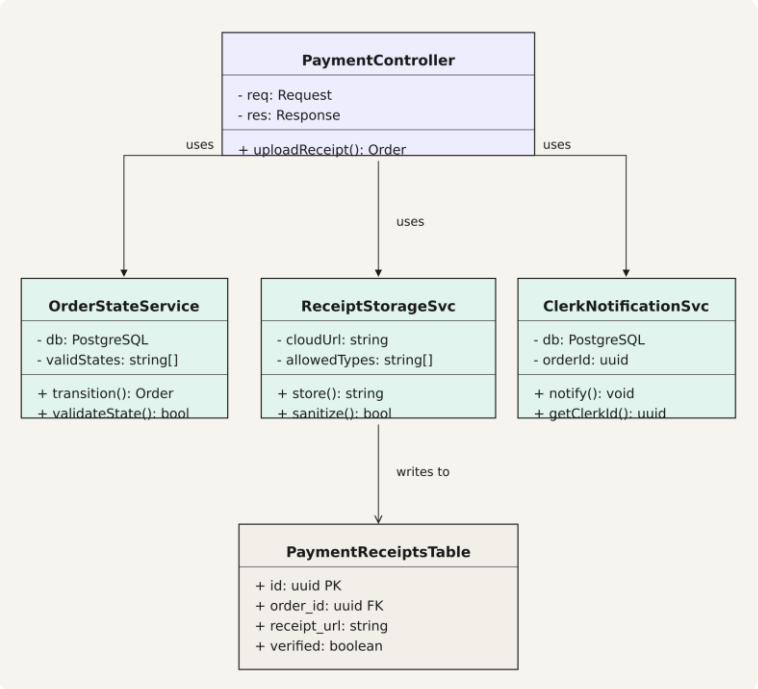
- Back-end component(s)
  - PaymentController
    - Description and purpose: The primary NestJS controller handling incoming POST requests to the `/orders/:id/receipt` endpoint. It receives the uploaded receipt image and the order ID, verifies the order is in the correct state (`PROOF_APPROVED`) before proceeding, and delegates receipt storage and state transition to the appropriate services. Returns the updated order record with state `AWAITING_PAYMENT_VERIFICATION` upon success.
    - Component type: NestJS `@Controller()` with `@Post()` decorator
  - OrderStateService

- Description and purpose: A NestJS injectable service responsible for validating and executing state transitions in the order lifecycle state machine. For this transaction it verifies the current order state is `PROOF_APPROVED` before allowing the transition to `AWAITING_PAYMENT_VERIFICATION`. Rejects the request with a 409 Conflict if the order is in an illegal state for this transition.
- Component type: NestJS `@Injectable()` service
  - ReceiptStorageService
- Description and purpose: A NestJS injectable service that handles the secure upload of the GCash receipt image to cloud storage. It sanitizes the file to ensure it is a flattened image (PNG/JPG only) with no embedded scripts or macros, generates a secure URL, and links the stored asset to the order ID in the PaymentReceiptsTable.
- Component type: NestJS `@Injectable()` service
  - ClerkNotificationService
- Description and purpose: A NestJS injectable service invoked after the receipt is successfully stored. It flags the assigned print shop clerk by creating a notification record linked to the order ID, alerting them that a payment receipt is awaiting manual verification on their dashboard.
- Component type: NestJS `@Injectable()` service
  - PaymentReceiptsTable
    - Description and purpose: The PostgreSQL table that persists all uploaded GCash receipt references. Each record stores the receipt ID, the linked order ID, the cloud storage URL of the receipt image, the upload timestamp, and a verification status flag

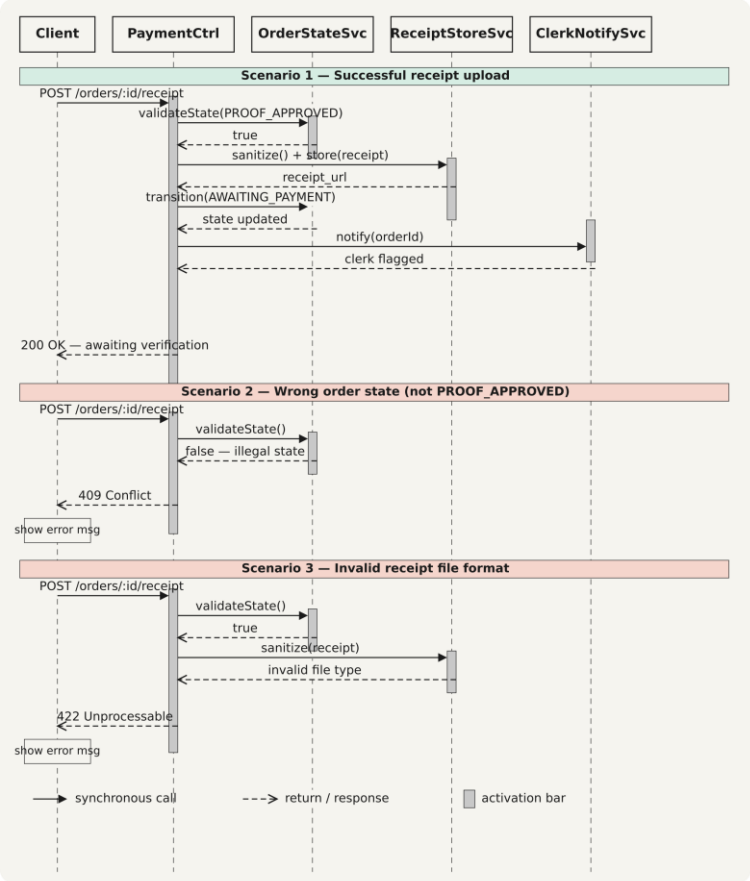
that the clerk updates upon manual review.

- Component type: PostgreSQL database table

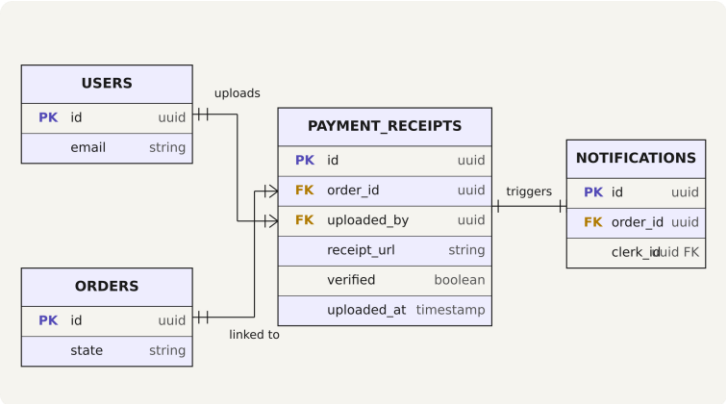
- Object-Oriented Components
  - Class Diagram



  - Sequence Diagram



- Data Design
  - ERD or schema



Module 3 – Asset Validation Constraint Module

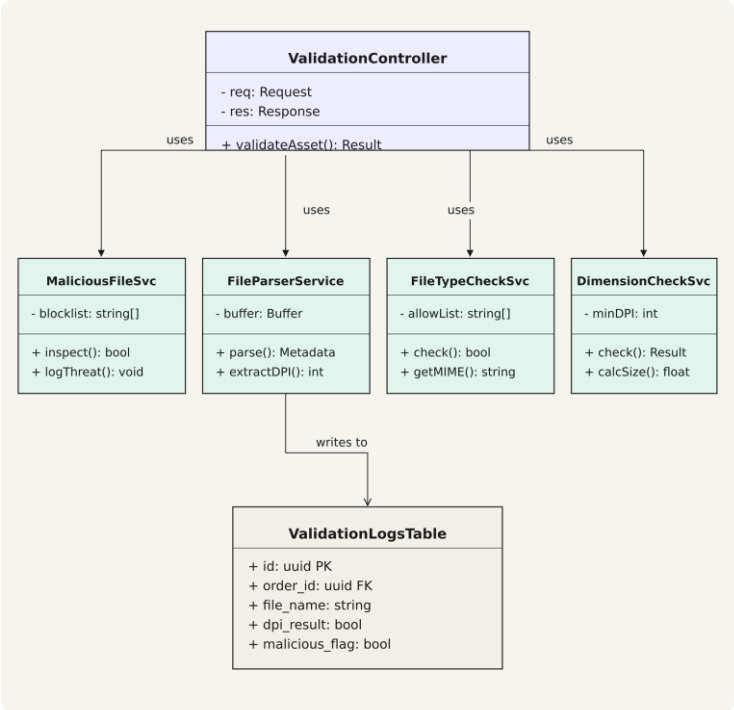
3.1 Automated Asset Verification

- User Interface Design
- Front-end component(s)
    - PreFlightModal
        - Description and purpose: The root modal component that overlays the New Order screen during file validation. It manages the local validation state object containing the current status of each check (file type, DPI, canvas size) and updates each row reactively as results stream in from the backend.
        - Component type: React Native modal component
    - FileTypeIndicator
        - Description and purpose: A status row component that displays the file type check result. Renders a green "Approved" label if the file is PDF or PNG, or a red "Rejected" label with the detected file type if it fails. Updates immediately upon metadata parsing.
        - Component type: React Native functional component
    - DPIChecker
        - Description and purpose: A status row component that displays the DPI validation result. Renders the detected DPI value alongside the minimum required DPI. Shows green if the threshold is met, or red with a specific failure message (e.g. "72 DPI — Minimum 300 DPI required") if it fails.
        - Component type: React Native functional component
    - CanvasSizeChecker
        - Description and purpose: A status row component that displays the canvas dimension compliance result. Compares the uploaded file's pixel dimensions against the selected paper size requirements. Renders green if compliant or red with the detected vs required dimensions if not.
        - Component type: React Native functional component
    - CancelUploadButton
        - Description and purpose: A pressable outlined button that aborts the in-progress validation and upload, clears the local file state, and returns the client to the file selection state in the New Order screen.
        - Component type: React Native TouchableOpacity
- Back-end component(s)
    - ValidationController

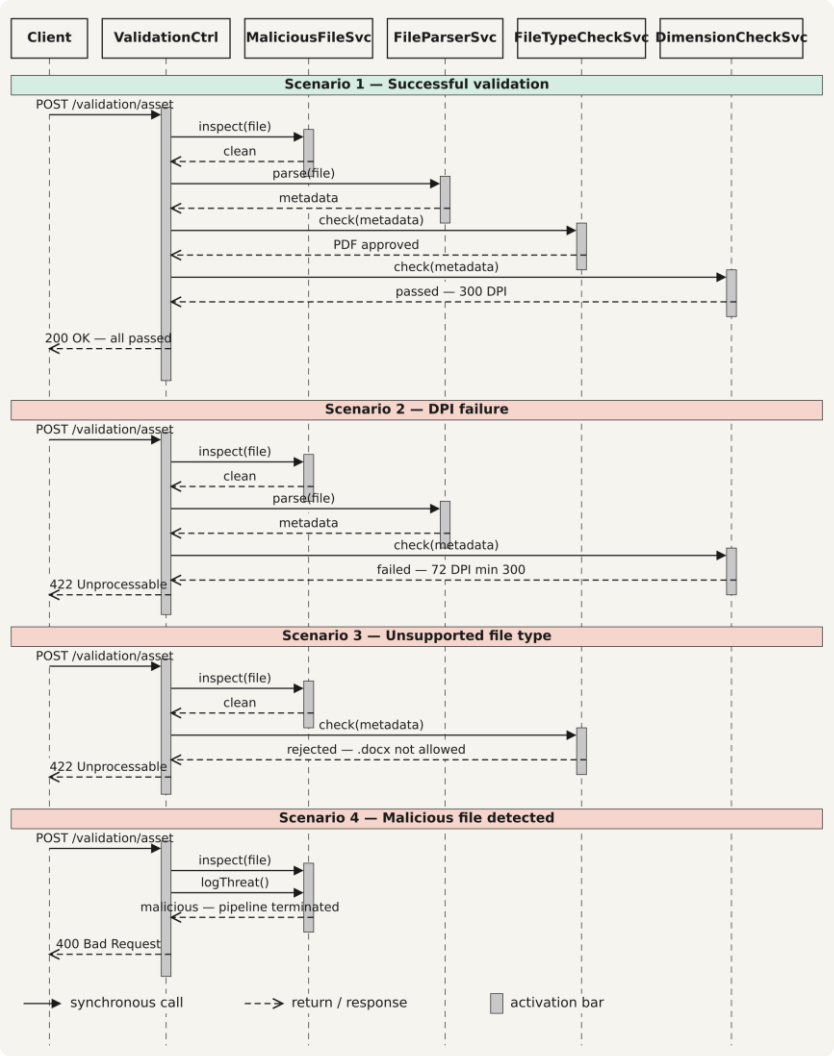
        - Description and purpose: The primary NestJS controller handling incoming POST requests to the `/validation/asset` endpoint. It receives the uploaded file, delegates the parsing and validation pipeline to the appropriate services in sequence, and returns a structured validation result object containing the status of each check (file type, DPI, canvas size) to the frontend for real-time display.
        - Component type: NestJS `@Controller()` with `@Post()` decorator
    - FileParserService

- Description and purpose: A NestJS injectable service that reads the raw binary metadata of the uploaded file. It extracts the file type, pixel dimensions, DPI, and color profile without fully loading the file into memory, enabling fast metadata-only parsing. Passes the extracted metadata object to the downstream validation services.
- Component type: NestJS `@Injectable()` service
    - FileTypeCheckService
- Description and purpose: A NestJS injectable service that receives the parsed file metadata and verifies the file extension and MIME type against the system's allowlist (PDF and PNG only). Explicitly rejects executable scripts, macro-embedded files, and any non-flattened formats. Returns a boolean result and the detected file type string.
- Component type: NestJS `@Injectable()` service
    - DimensionCheckService
- Description and purpose: A NestJS injectable service that receives the parsed pixel dimensions and DPI from the FileParserService and mathematically verifies compliance against the selected print shop's paper size constraints. Calculates whether the physical print size at the given DPI meets the minimum 300 DPI threshold and the required canvas dimensions. Returns a structured result containing the detected values and the required values for display.
- Component type: NestJS `@Injectable()` service
    - MaliciousFileDetectorService
- Description and purpose: A NestJS injectable service that performs a deep inspection of the uploaded file to detect embedded scripts, macros, or executable payloads before any further processing occurs. Acts as the first gate in the validation pipeline. If a malicious payload is detected, it immediately terminates the pipeline and logs the attempt to the ValidationLogsTable.
- Component type: NestJS `@Injectable()` service
    - ValidationLogsTable
        - Description and purpose: The PostgreSQL table that persists all asset validation attempts and their results. Each record stores the log ID, the linked order ID, the file name, the validation result for each check (file type, DPI, canvas size), a malicious file flag, and the validation timestamp.
        - Component type: PostgreSQL database table

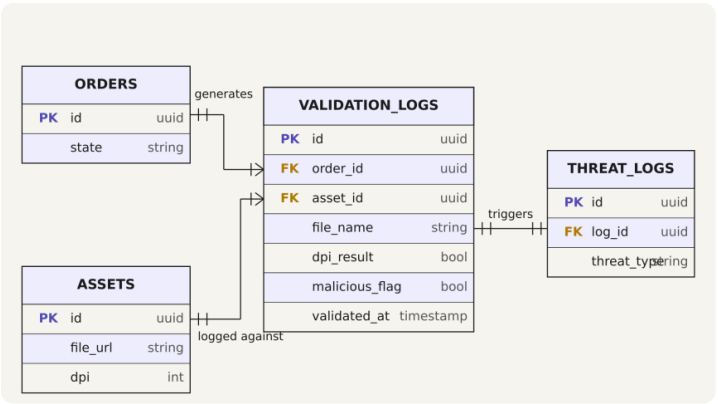
- Object-Oriented Components
    - Class Diagram



- Sequence Diagram



- Data Design
  - ERD or schema



3.2 Generate Immutable Proof Agreement

- User Interface Design
- Front-end component(s)
    - ProofViewer
        - Description and purpose: The primary display component that renders the low-resolution web proof of the validated asset inside a centered preview panel. It fetches the proof image URL from the backend and displays it in a contained viewport. The proof is rendered in read-only mode — no editing, zooming, or downloading is permitted from this component.
        - Component type: React Native Image component
    - OrderDetailPanel

▪ Description and purpose: A side panel component that displays the metadata associated with the current order — order ID, selected print shop, paper size, and pricing total. Also displays a warning label informing the client that approving the proof locks the file permanently and cannot be undone.

▪ Component type: React Native functional component

- ApproveContractButton

▪ Description and purpose: A full-width pressable button that dispatches the proof approval request to the ProofController endpoint. On press it displays a confirmation dialog warning the client that this action is irreversible before proceeding. Enters a loading state during the API call and is permanently disabled after a successful approval to prevent double submission.

▪ Component type: React Native TouchableOpacity

- IrreversibleConfirmDialog

▪ Description and purpose: A modal dialog triggered by the ApproveContractButton that presents the client with a final confirmation prompt before locking the proof. Contains a confirm and a cancel option. Only on confirmation does it dispatch the approval request.

▪ Component type: React Native Modal component

- Component Name

▪ Description and purpose

▪ Component type or format

- Component Name
    - Description and purpose
    - Component type or format

- Back-end component(s)
    - ProofController
        - Description and purpose: The primary NestJS controller handling two endpoints — a

GET request to `/orders/:id/proof` to fetch the rendered proof for display, and a POST request to `/orders/:id/proof/approve` to trigger the approval and lock sequence. Delegates rendering and locking logic to the appropriate services and returns the updated order record upon successful approval.

- Component type: NestJS `@Controller()` with `@Get()` and `@Post()` decorators

- ProofRendererService

- Description and purpose: A NestJS injectable service that generates a low-resolution web-safe rendering of the validated asset stored in cloud storage. It fetches the original high-resolution file, downsamples it to a web-safe resolution for preview purposes, and returns a temporary signed URL for the frontend to display. The original high-resolution file remains untouched in cloud storage.
- Component type: NestJS `@Injectable()` service

- ProofLockService

- Description and purpose: A NestJS injectable service invoked upon client approval. It writes a cryptographic lock record to the ProofsTable containing the approving User ID, the exact timestamp of approval, and a hash of the file state at the time of approval. Once written, the record cannot be modified or deleted, ensuring 100% unalterable data retention for future dispute resolution.
- Component type: NestJS `@Injectable()` service

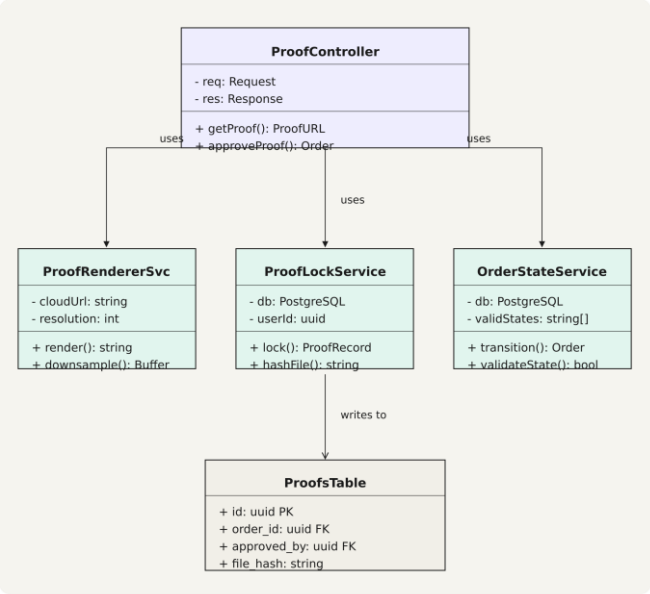
- OrderStateService

- Description and purpose: Reused from Transaction 2.2. In this context it validates that the order is in a legal state for proof approval (must be in `OPEN_INQUIRY`) and transitions the order state to `PROOF_APPROVED` upon successful lock record creation.
- Component type: NestJS @Injectable() service

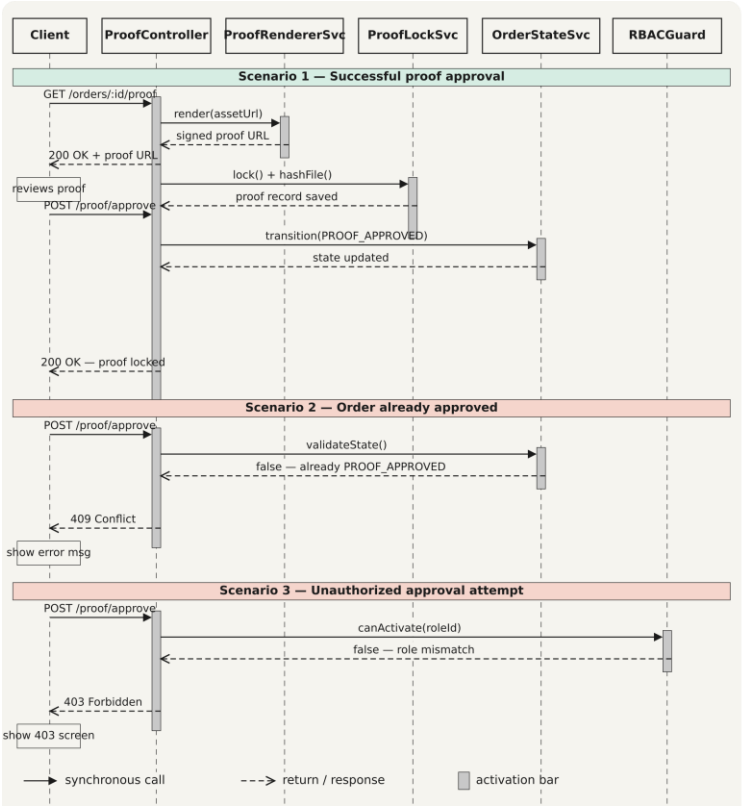
- ProofsTable

- Description and purpose: The PostgreSQL table that persists all proof approval records. Each record stores the proof ID, the linked order ID, the approving User ID, the file hash at the time of approval, the proof image URL, and the approval timestamp. No delete or update operations are permitted on this table.
- Component type: PostgreSQL database table

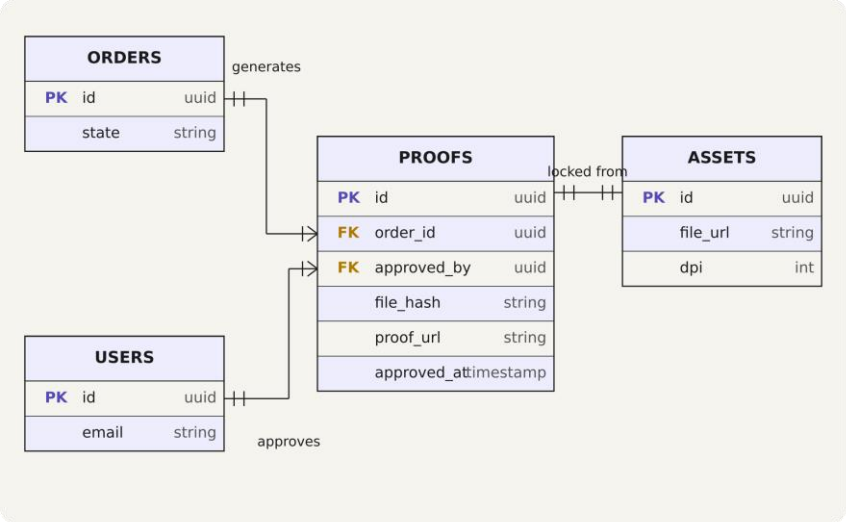
- Object-Oriented Components
  - Class Diagram



- Sequence Diagram



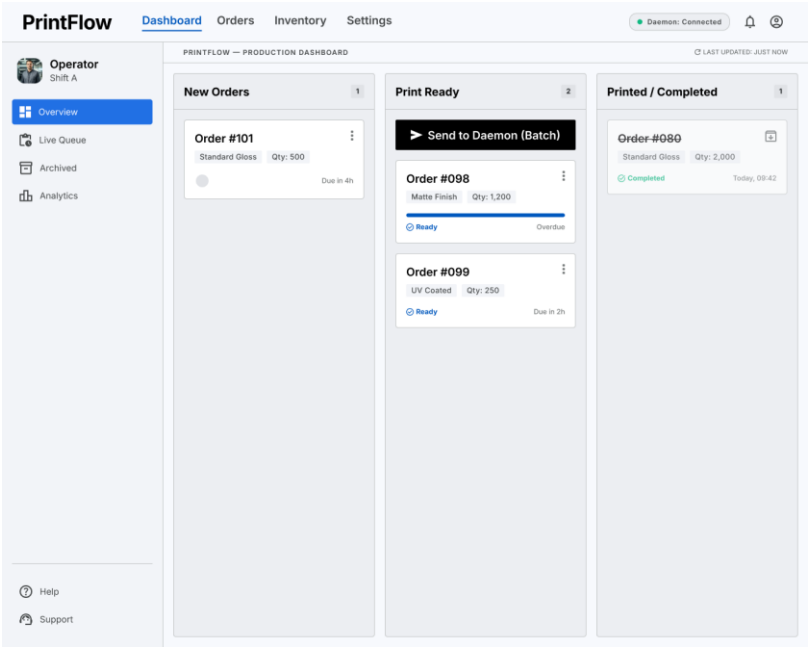
- Data Design
  - ERD or schema



Module 4 – Edge Execution and Fulfillment Ledger

4.1 Automated Edge Print Spooling

- User Interface Design



- Front-end component(s)
    - ProductionQueueScreen

        - **Description and purpose:** The root screen for print shop clerks managing physical fulfillment. Renders the full Kanban-style production board with three columns: New Orders, Print Ready, and Printed/Completed. Serves as the entry point for selecting orders and initiating a batch print execution command.
        - **Component type:** React Native Screen
    - PrintReadyColumn
        - **Description and purpose:** Renders the scrollable list of order cards currently in the IN_PRODUCTION state and eligible for batch spooling. Provides a select-all checkbox at the column header level to allow clerks to group multiple jobs into a single batch dispatch.
        - **Component type:** React Native functional component
    - OrderCard
        - **Description and purpose:** Individual card representing one print job within the production queue. Displays the order ID, client name, asset filename, paper size, and a color-coded state badge. Includes a checkbox for individual selection into the current batch group.
        - **Component type:** React Native TouchableOpacity
    - BatchExecuteButton
        - **Description and purpose:** A prominent action button fixed at the top of the Print Ready column. On press it dispatches the batch spooling command to the EdgeDaemonGateway via the DaemonSocketService. Renders as disabled when no orders are selected or when the DaemonStatusBadge reports an offline state, preventing accidental dispatch.
        - **Component type:** React Native TouchableOpacity
    - DaemonStatusBadge
        - **Description and purpose:** A persistent indicator anchored in the dashboard header that displays the real-time WebSocket connection state of the shop's local Edge Daemon. Cycles between three visible states: Connected, Reconnecting, and Offline. Driven by connection lifecycle events emitted by the DaemonSocketService.
        - **Component type:** React Native functional component
    - PrintJobProgressModal
        - **Description and purpose:** A modal overlay triggered immediately after the clerk initiates a batch print. Displays a per-job status list updated in real time as the daemon processes each asset through the pipeline: Queued, Downloading, Sent to Spooler, and Failed. Remains open until the clerk dismisses it or all jobs resolve.
        - **Component type:** React Native Modal component
    - DaemonOfflineFallbackPanel
        - **Description and purpose:** A conditionally rendered panel that replaces the BatchExecuteButton area when the daemon connection is offline. Informs the clerk that automated spooling is unavailable and surfaces a manual download button per order, allowing physical production to continue without the daemon as graceful

degradation.

- **Component type:** React Native functional component

- DaemonSocketService
  - **Description and purpose:** A client-side Socket.io service that maintains the persistent WSS connection to the NestJS EdgeDaemonGateway. Listens for daemon acknowledgment events and job status updates, then propagates connection state changes to the DaemonStatusBadge and per-job progress updates to the PrintJobProgressModal.
  - **Component type:** Service
- OrderStateService
  - **Description and purpose:** Client-side service that interfaces with the backend REST API to fetch and update order states. Reused from prior transactions and extended here to handle the IN_PRODUCTION to PRINTED state transition upon confirmed daemon acknowledgment.
  - **Component type:** Service

- Back-end component(s)
  - ProductionQueueController
    - **Description and purpose:** A NestJS REST controller that exposes endpoints for the clerk-facing production dashboard. Handles requests to fetch all orders in the IN_PRODUCTION state filtered by the authenticated shop's tenant ID. Delegates state transition writes to the OrderStateService.
    - **Component type:** NestJS `@Controller()` with `@Get()` decorator
  - EdgeDaemonGateway

- **Description and purpose:** A NestJS WebSocket Gateway that maintains the persistent WSS connection between the cloud backend and each shop's local Edge Daemon. Receives the batch execute command from the clerk, constructs the secure job payload containing signed asset URLs, and emits it to the correct shop-scoped daemon socket. Also listens for per-job acknowledgment events from the daemon and forwards status updates to the connected clerk client.
- **Component type:** NestJS `@WebSocketGateway()` with `@SubscribeMessage()` decorator
  - PrintSpoolerService
- **Description and purpose:** A NestJS injectable service that orchestrates the full batch spooling workflow. Validates that each selected order is in the IN_PRODUCTION state, assembles the signed asset URL payload, delegates the WebSocket emission to the EdgeDaemonGateway, and coordinates with the OrderStateService to transition each order to PRINTED upon successful daemon acknowledgment. Also intercepts malicious or non-whitelisted payload types before dispatch.
- **Component type:** NestJS `@Injectable()` Service
  - OrderStateService
- **Description and purpose:** A reused NestJS injectable service responsible for all order state transition writes to the PostgreSQL database. Extended here to handle the IN_PRODUCTION to PRINTED transition and

to enforce the constraint that only orders in the correct prior state may advance, preserving the integrity of the state machine.

- **Component type:** NestJS `@Injectable()` Service
    - PrintJobsRepository

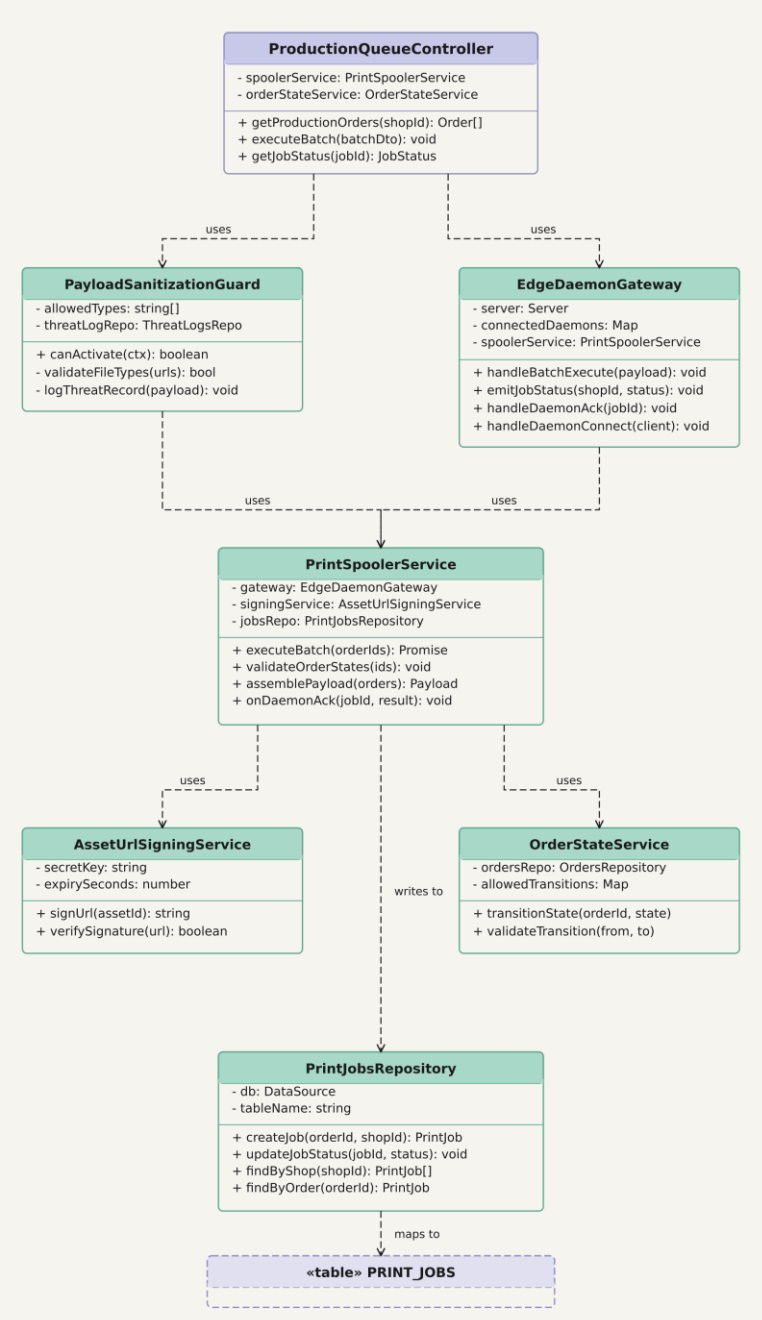
- **Description and purpose:** A NestJS injectable repository that handles all database read and write operations against the PRINT_JOBS table. Creates a new print job record per order at the moment of batch dispatch, writes daemon acknowledgment results, and exposes query methods for job status lookups by order ID and shop ID.

- **Component type:** NestJS `@Injectable()` Repository
    - PayloadSanitizationGuard
        - **Description and purpose:** A NestJS Guard applied to the batch execute endpoint that intercepts the outbound file payload before the WebSocket emission. Validates that every asset URL in the batch resolves to a whitelisted, flattened file type (PDF or PNG only). Rejects the entire batch and logs a threat record if any asset fails the sanitization check, preventing the Edge Daemon from acting as a malware vector into the shop's local network.
        - **Component type:** NestJS `@Injectable()` implementing `CanActivate`
    - AssetUrlSigningService

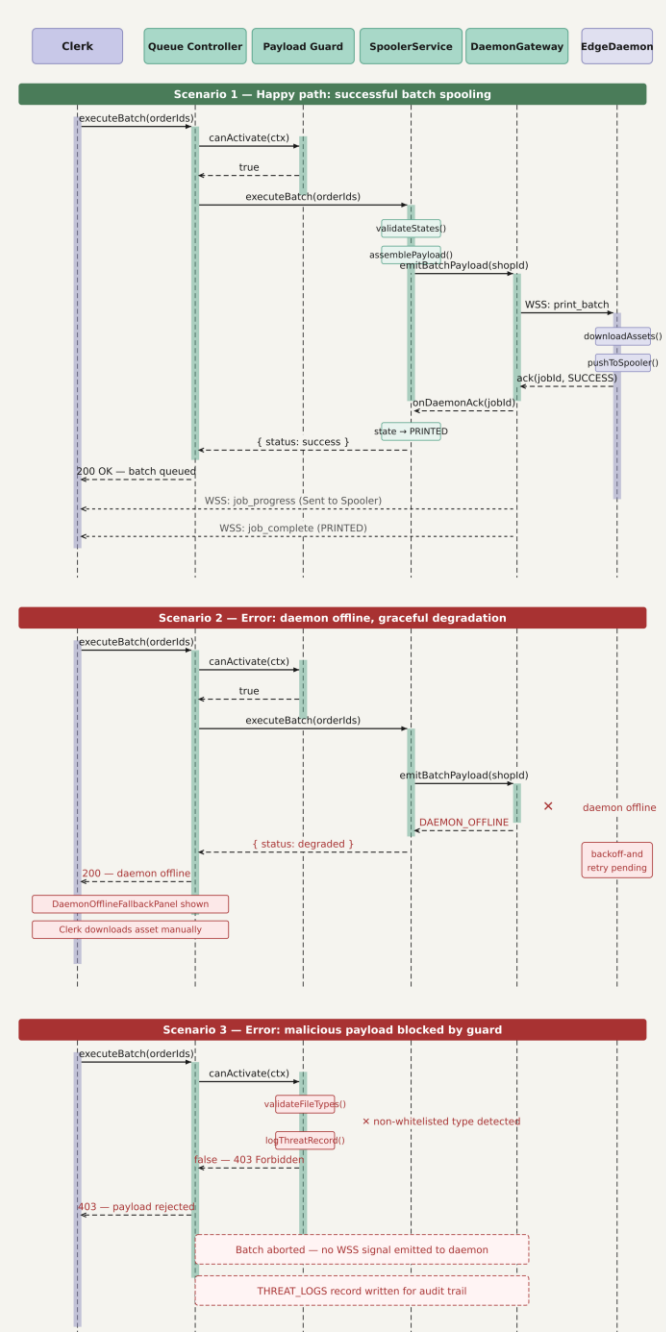
- **Description and purpose:** A NestJS injectable service that generates short-lived, cryptographically signed URLs for each validated asset in the batch. The signed URLs are embedded in the WebSocket payload sent to the daemon, ensuring that only the intended daemon instance can download the files within the allowed time window.

- **Component type:** NestJS `@Injectable()` Service
    - PrintJobsModule
        - **Description and purpose:** The NestJS module that encapsulates all components related to edge print spooling. Declares and exports the EdgeDaemonGateway, PrintSpoolerService, PrintJobsRepository, PayloadSanitizationGuard, and AssetUrlSigningService. Imports the OrdersModule to access the shared OrderStateService.
        - **Component type:** NestJS `@Module()` decorator

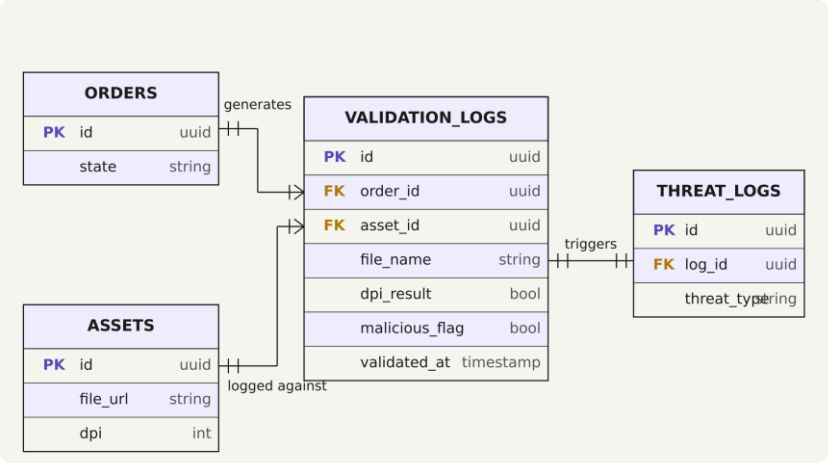
- Object-Oriented Components
    - Class Diagram



- Sequence Diagram

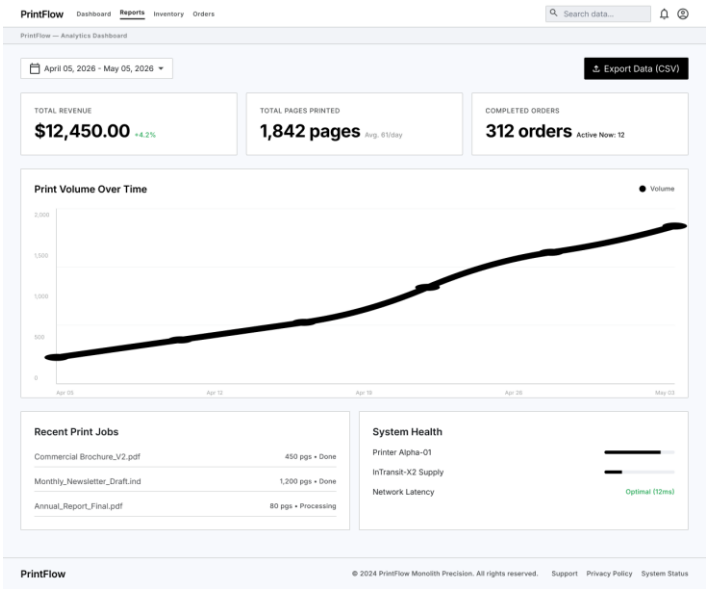


- Data Design
    - ERD or schema



4.2 Extract Operations and Revenue Analytics

- User Interface Design



- Front-end component(s)
    - AnalyticsDashboard
        - Description and purpose: The root dashboard component that manages the selected date range state and coordinates data fetching across all child components. On date range change it dispatches a new query to the AnalyticsController and distributes the returned aggregate data to the KPI cards and line chart.
        - Component type: React Native functional component
    - DateRangePicker

▪ Description and purpose: A dropdown selector component that allows the manager to define a custom reporting period by selecting a start and end date. On selection it updates the AnalyticsDashboard date range

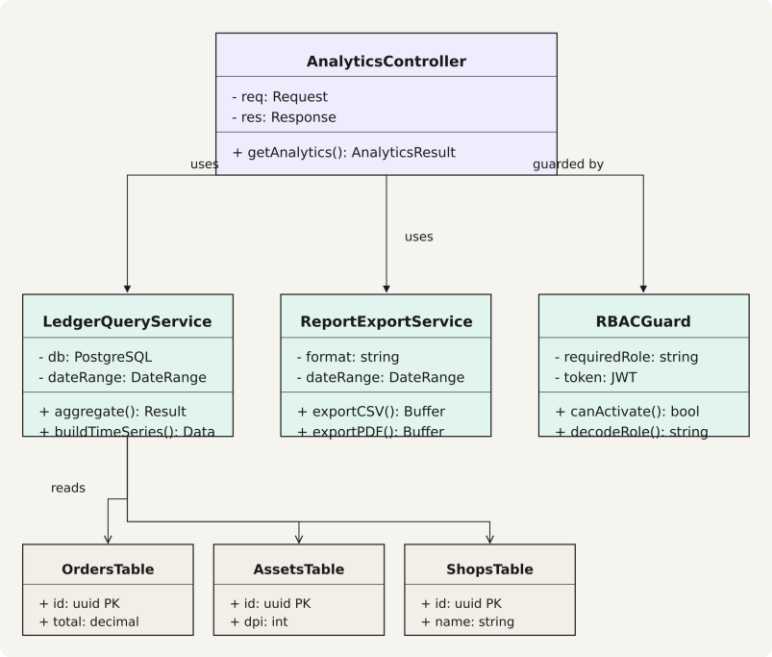
state and triggers a new data fetch. Defaults to the current academic month on initial render.

- Component type: React Native functional component
    - KPICard
- Description and purpose: A reusable card component that displays a single aggregate metric. Accepts a label and a value as props and renders them in a minimal white card with a thin border. Used three times on the dashboard for Total Revenue, Total Pages Printed, and Completed Orders.
- Component type: React Native functional component
    - VolumeLineChart
- Description and purpose: A full-width chart component that renders the print volume over time as a line graph for the selected date range. Receives the time-series data from the AnalyticsDashboard and plots it with dates on the x-axis and volume on the y-axis. Displays a flat empty state with a "No data for selected period" label if the query returns zero records.
- Component type: React Native functional component
    - ExportButton
- Description and purpose: A pressable button positioned in the top controls row that dispatches an export request to the ReportExportService endpoint, passing the current date range and aggregate data as parameters. Supports both CSV and PDF export formats selectable via a small dropdown. Enters a loading state during file generation and triggers a download on completion.
- Component type: React Native TouchableOpacity

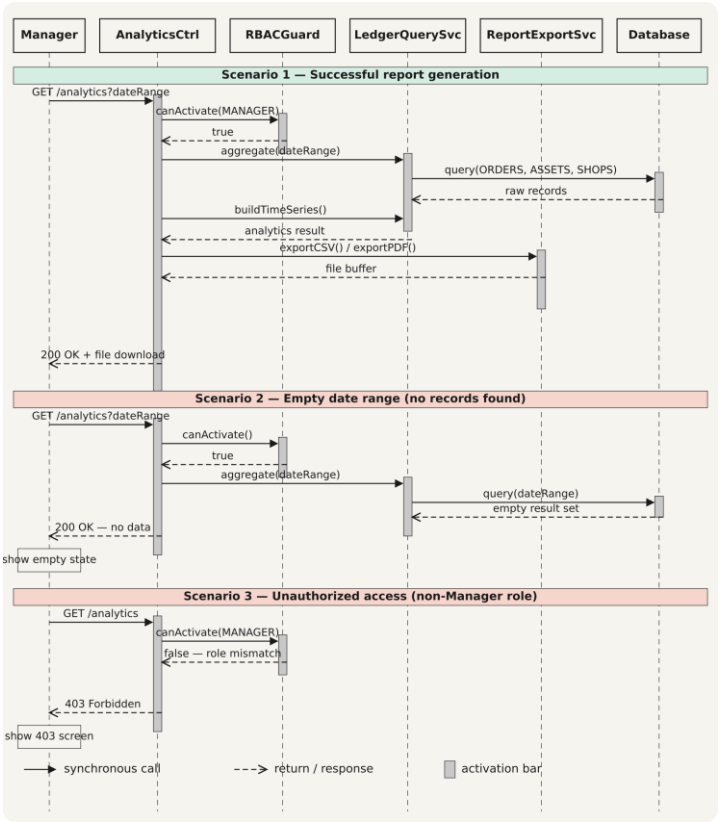
- Back-end component(s)
    - AnalyticsController
        - Description and purpose: The primary NestJS controller handling incoming GET requests to the `/analytics` endpoint. It receives the date range parameters from the manager's dashboard, delegates data aggregation to the LedgerQueryService, and returns the structured aggregate metrics object containing revenue totals, page counts, completed order counts, and time-series volume data for rendering on the dashboard.Description and purpose: The primary NestJS controller handling incoming GET requests to the `/analytics` endpoint. It receives the date range parameters from the manager's dashboard, delegates data aggregation to the LedgerQueryService, and returns the structured aggregate metrics object containing revenue totals, page counts, completed order counts, and time-series volume data for rendering on the dashboard.
        - Component type: NestJS `@Controller()` with `@Get()` decorator
    - LedgerQueryService
- Description and purpose: A NestJS injectable service that queries the immutable fulfillment ledger by reading from the ORDERS, ASSETS, and SHOPS tables within the specified date range. It aggregates total revenue by summing order totals, calculates total pages printed from asset records, counts completed orders, and builds a time-series dataset of print volume grouped by date for the line chart. Returns a structured analytics result object.
- Component type: NestJS `@Injectable()` service
    - ReportExportService

- Description and purpose: A NestJS injectable service invoked by the ExportButton on the frontend. It receives the current date range and the aggregated metrics object, formats the data into either a CSV or PDF document depending on the requested format, and returns the file as a downloadable binary stream. For PDF generation it uses a server-side PDF library to render a formatted report with the shop name, date range, KPI values, and a tabular breakdown of completed orders.
- Component type: NestJS `@Injectable()` service
    - OrderStateService
- Description and purpose: Reused from Transaction 2.2. In this context it validates that the order is in a legal state for proof approval (must be in `OPEN_INQUIRY`) and transitions the order state to `PROOF_APPROVED` upon successful lock record creation.
- Component type: NestJS @Injectable() service
    - RBACGuard
        - Description and purpose: Reused from Transaction 1.2. In this context it enforces that only users with the Manager role can access the `/analytics` endpoint. Any request from a Clerk or Student role is rejected with a 403 Forbidden response before reaching the AnalyticsController
        - Component type: NestJS `@Injectable()` Guard implementing `CanActivate`

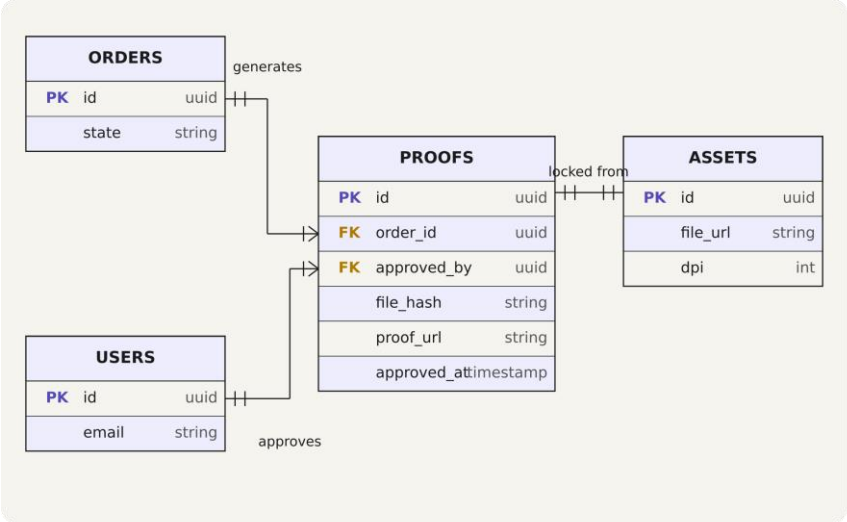
- Object-Oriented Components
    - Class Diagram



- Sequence Diagram



- Data Design
  - ERD or schema


ORDERS
PK id uuid
state string
generates
PROOFS
PK id uuid
FK order_id uuid
FK approved_by uuid
file_hash string
proof_url string
approved_at timestamp
locked from
ASSETS
PK id uuid
file_url string
dpi int
USERS
PK id uuid
email string
approves